

OASIS INFOBYTE
DATA ANALYTICS INTERNSHIP
LEVEL 2 – TASK 5
AUTOCOMPLETE AND AUTOCORRECT DATA ANALYTICS

Final Project Report

Submitted By

Madala Sai Sprisha

Internship

OASIS INFOBYTE – Data Analytics Internship

Domain

Data Analytics

Project Title

Level 2 – Task 5: Autocomplete and Autocorrect Data Analytics

Technologies Used

Python

Pandas

NumPy

NLTK

Pyspellchecker

Collections

Matplotlib

Jupyter Notebook

Submitted To

OASIS INFOBYTE

Year

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to OASIS INFOBYTE for providing me with the opportunity to complete Level 2 – Task 5, Autocomplete and Autocorrect Data Analytics. This project provided practical exposure to natural language processing, text preprocessing, n-gram modelling, next-word prediction, spelling correction, evaluation metrics, visualization, and comparative analysis.

I also thank the mentors and coordinators for providing the internship task requirements and learning resources. The structured workflow helped me understand how a large text corpus can be cleaned and transformed into a useful language-processing dataset and how NLP models can be evaluated using measurable performance indicators.

The implementation was completed in Python using a large email corpus. The workflow begins with library import and dataset inspection, continues through tokenization, lowercasing, punctuation removal and stopword filtering, and then implements frequency-based autocomplete and autocorrect. The final stages evaluate the models and compare bigram and trigram approaches.

This work strengthened my practical understanding of NLP workflows and demonstrated how simple frequency-based language models can be used to predict the next word and correct deliberately misspelled words.

ABSTRACT

This project focuses on implementing and evaluating autocomplete and autocorrect functionality using natural language processing techniques on a large email text corpus. The executed notebook loads 517,401 records with two columns: file and message. The preprocessing workflow retains valid messages, converts text to lowercase, removes punctuation, tokenizes the messages, removes English stopwords, and produces processed text suitable for language modelling.

A frequency-based bigram model is constructed from the cleaned corpus. The model contains 1,651,227 words with available next-word predictions and is tested on ten input prefixes with three predictions per prefix. The selected tests achieve 100% Top-1 accuracy and 100% Top-3 accuracy.

For autocorrect, twenty deliberately misspelled words are evaluated using a vocabulary-based spelling correction workflow. The final test set achieves 100% autocorrect accuracy, 100% precision, and 100% recall. A confusion matrix confirms that all twenty test corrections fall on the correct diagonal.

The project also compares bigram and trigram models using a 20,000-email sample. The comparison demonstrates that trigrams use a larger context and therefore can represent more context-specific word sequences. A top-20 word frequency chart and final performance summary provide additional evidence for the completed implementation.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	Cover Page	1
	Acknowledgement	2
	Abstract	3
	Table of Contents	4–5
Chapter 1	Introduction	
1.1	Project Overview	6
1.2	Problem Statement	6
1.3	Project Objectives	6
Chapter 2	Dataset & Tools	
2.1	Dataset Description	7
2.2	Tools & Technologies Used	7
2.3	Project Folder Structure	8
2.4	Input Dataset	8
Chapter 3	Autocomplete & Autocorrect Implementation	
3.1	Library Import & Dataset Loading	9
3.2	Dataset Inspection & Quality Assessment	10
3.3	NLP Preprocessing	11
3.4	Frequency-Based Bigram Model	12
3.5	Autocomplete Prediction	13
3.6	Autocomplete Testing on 10 Prefixes	14
3.7	Autocorrect Implementation	15
3.8	Autocorrect Testing on 20 Words	15

3.9	Autocorrect Accuracy	16
3.10	Autocomplete Performance Metrics	17
3.11	Autocorrect Precision & Recall	18
3.12	Bigram vs Trigram Comparison	19
3.13	Top 20 Most Frequent Words	20
3.14	Autocorrect Confusion Matrix	21
3.15	Final Performance Summary	22
3.16	Discussion and Limitations	23
Chapter 4	Results & Discussion	
4.1	Key Findings	24
4.2	Data-Driven Insights	25
4.3	Conclusion	25
4.4	Future Enhancements	25
4.5	References	26
4.6	Final Findings and Conclusion	26

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Autocomplete and Autocorrect Data Analytics project focuses on applying natural language processing techniques to a large email corpus. The implementation builds a frequency-based bigram language model to predict likely next words and a spelling-correction workflow to identify suitable corrections for deliberately misspelled words.

The project combines text preprocessing, vocabulary construction, frequency counting, next-word prediction, spelling correction, evaluation metrics, and visualization. The implementation is designed as a practical demonstration of how basic NLP components can be combined into an end-to-end text prediction and correction system.

1.2 Problem Statement

Typing systems frequently need to predict what a user is likely to type next and correct words that contain spelling errors. Raw text contains punctuation, inconsistent capitalization, stopwords, metadata, and other noise that can reduce the usefulness of a language model. The project therefore aims to prepare a large text corpus and build measurable autocomplete and autocorrect components.

1.3 Project Objectives

1. Collect and load a large text corpus suitable for NLP analysis.
2. Inspect the dataset structure, missing values, and duplicate records.
3. Apply lowercasing, punctuation removal, tokenization, and stopwords removal.
4. Build a frequency-based bigram model for next-word prediction.
5. Test autocomplete on at least 10 input prefixes and return the top 3 predictions.
6. Implement spelling correction using a vocabulary-based matching approach.
7. Test autocorrect on 20 deliberately misspelled words.
8. Calculate autocomplete Top-1 and Top-3 accuracy.
9. Calculate autocorrect accuracy, precision, and recall.
10. Compare bigram and trigram approaches on a manageable text sample.
11. Visualize the most frequent words and autocorrect confusion matrix.
12. Discuss the limitations of the implementation compared with production typing systems.

CHAPTER 2: DATASET & TOOLS

2.1 Dataset Description

The implementation uses an email text corpus loaded from emails.csv. The executed notebook reports 517,401 records and two columns: file and message. The file field identifies the source email path, while the message field contains the email text used for NLP preprocessing.

The quality inspection reports zero missing values in both columns and zero duplicate rows. The large size of the corpus provides substantial text for frequency-based language modelling.

2.2 Tools & Technologies Used

TOOL / TECHNOLOGY	PURPOSE
Python	Programming language and complete analytics workflow
Pandas	Dataset loading, cleaning, transformation, and tabular analysis
NumPy	Numerical operations and supporting calculations
NLTK	Tokenization and English stopword resources
pyspellchecker	Vocabulary-based spelling correction
collections	Counter and defaultdict for n-gram frequency modelling
Matplotlib	Analytical charts and confusion-matrix visualization
Jupyter Notebook / VS Code	Interactive implementation, execution, and documentation

2.3 Project Folder Structure

The implementation is organized around a dataset area containing emails.csv, a Jupyter Notebook containing the complete analysis, and a screenshots area used to preserve implementation evidence. The notebook reads the dataset through a relative path and records the outputs of each major NLP stage for reporting.

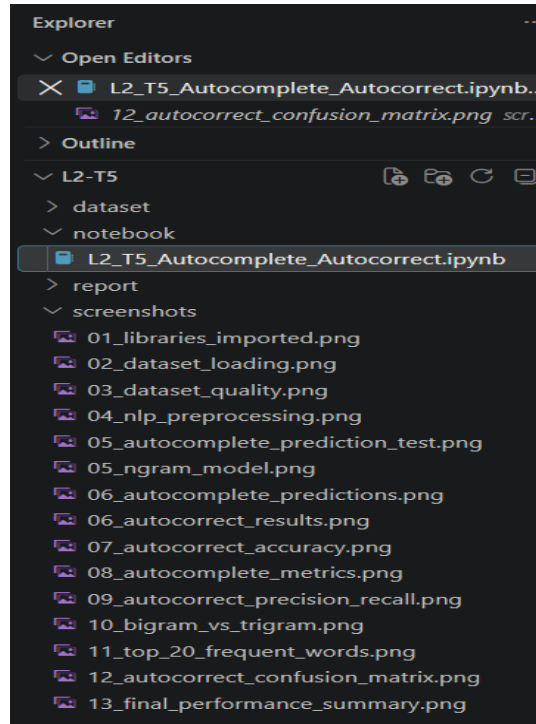


Figure 2.1: Project Folder Structure

2.4 Input Dataset

The input dataset contains 517,401 rows and two columns. The initial preview shows file paths and corresponding email messages. The dataset inspection confirms that the message field is available for preprocessing and that the dataset contains no missing values or duplicate rows at the initial quality stage.

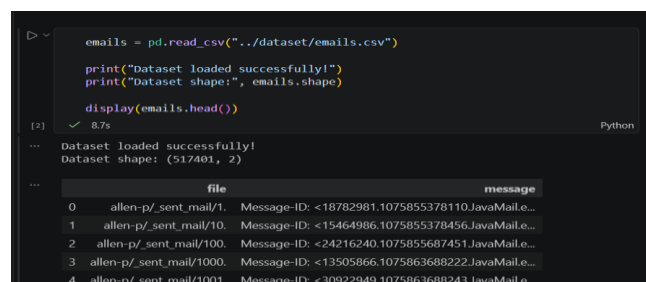


Figure 2.1: Dataset Loading

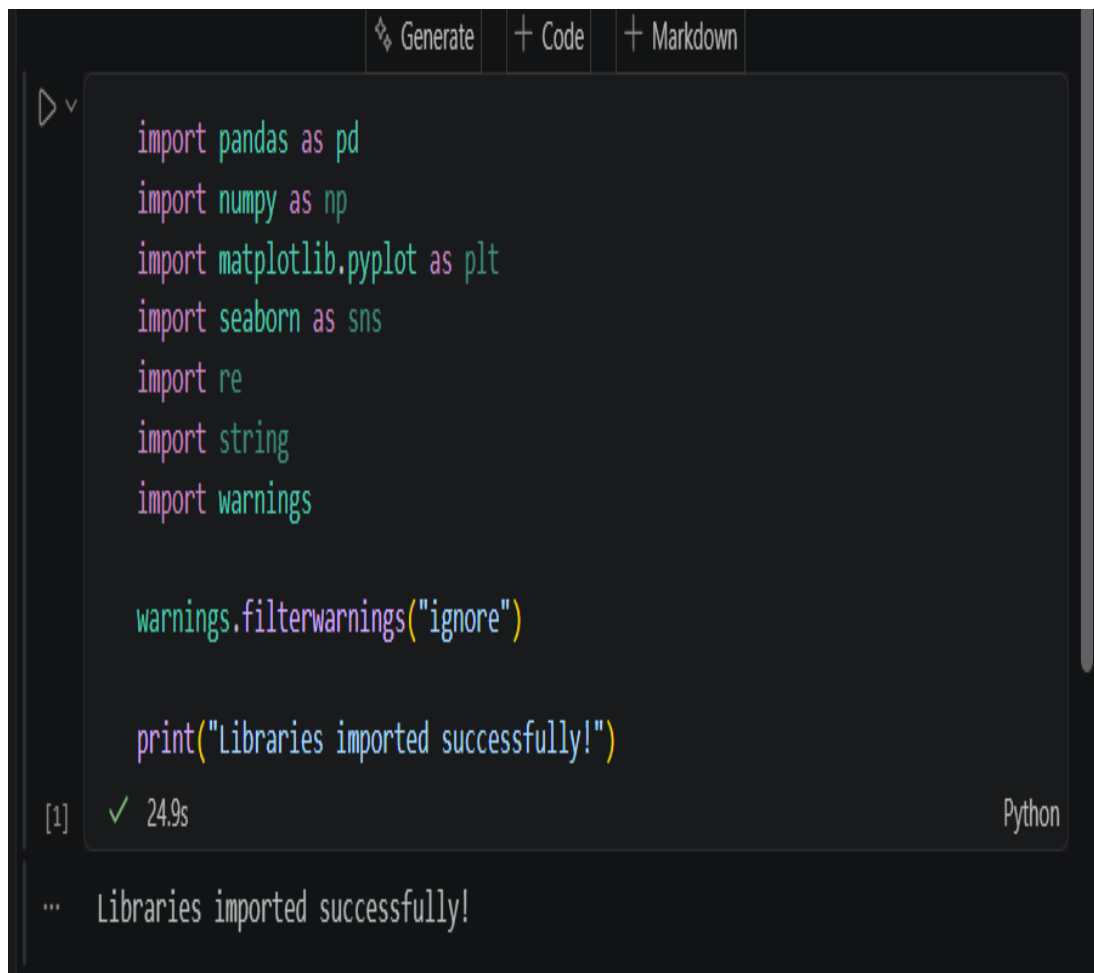
Interpretation:

The dataset was loaded successfully with 517,401 records and two columns, file and message, providing the input corpus for the NLP workflow.

CHAPTER 3: AUTOCOMPLETE & AUTOCORRECT IMPLEMENTATION

3.1 Library Import & Dataset Loading

The required Python libraries were imported successfully, including Pandas, NumPy, Matplotlib, Seaborn, regular-expression utilities, NLTK, and collections. The dataset was then loaded using `pd.read_csv()`, producing a DataFrame with 517,401 records and two columns, file and message.



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import string
import warnings

warnings.filterwarnings("ignore")

print("Libraries imported successfully!")
```

[1] ✓ 24.9s Python

... Libraries imported successfully!

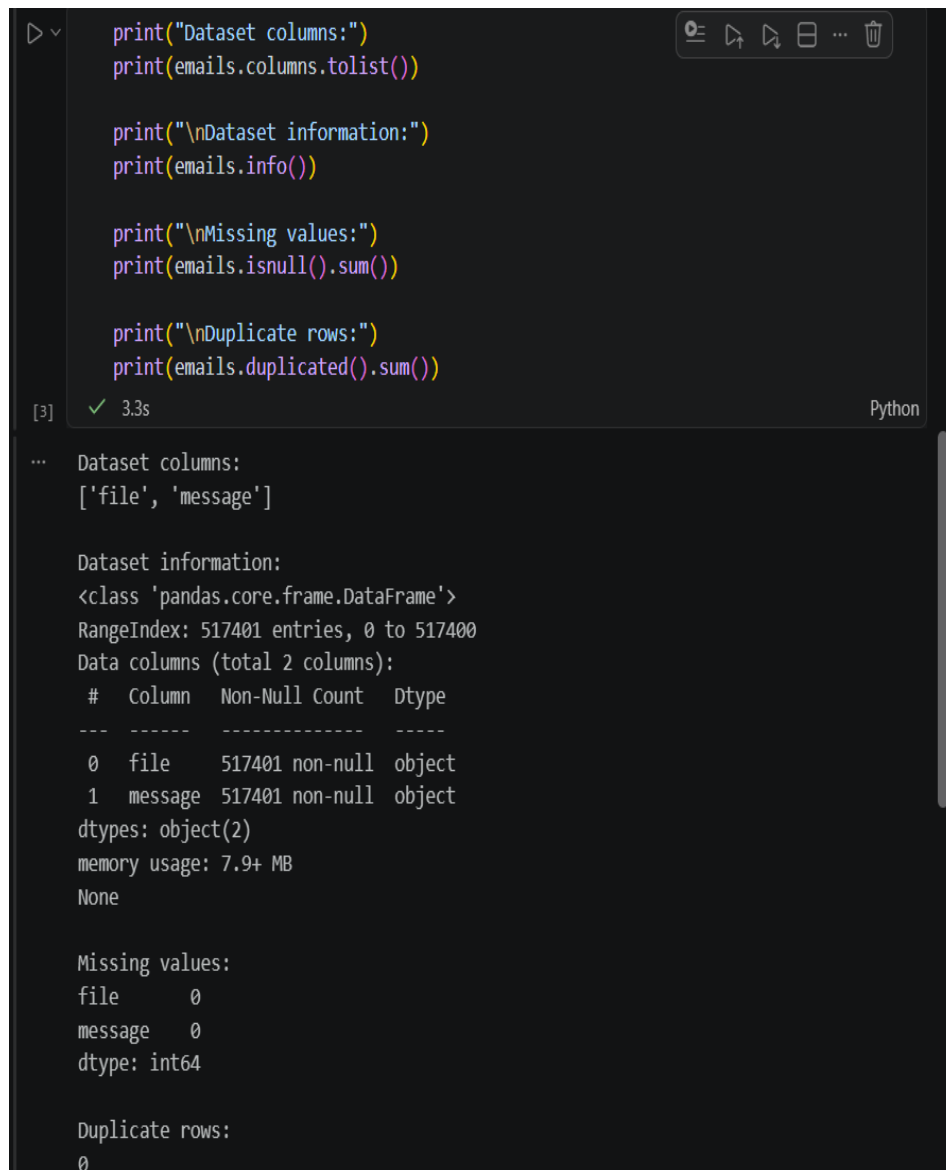
Figure 3.1: Libraries Imported Successfully

Interpretation:

The required Python libraries were imported successfully, including Pandas, NumPy, Matplotlib, Seaborn, regular-expression utilities, NLTK, and collections. The dataset was then loaded using `pd.read_csv()`, producing a DataFrame with 517,401 records and two columns.

3.2 Dataset Inspection & Quality Assessment

The notebook checks the dataset columns, data types, missing values, and duplicate rows. The two fields are file and message, both stored as object data types. The quality assessment reports zero missing values and zero duplicate rows.



```
print("Dataset columns:")
print(emails.columns.tolist())

print("\nDataset information:")
print(emails.info())

print("\nMissing values:")
print(emails.isnull().sum())

print("\nDuplicate rows:")
print(emails.duplicated().sum())
```

[3] ✓ 3.3s Python

```
... Dataset columns:
['file', 'message']

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 517401 entries, 0 to 517400
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   file        517401 non-null object
1   message     517401 non-null object
dtypes: object(2)
memory usage: 7.9+ MB
None

Missing values:
file      0
message   0
dtype: int64

Duplicate rows:
0
```

Figure 3.2: Dataset Inspection and Quality Assessment

Interpretation:

The results confirm that the dataset is suitable for the next processing stage because no missing values or duplicate rows were detected. The two fields, file and message, are stored as object data types.

3.3 NLP Preprocessing

The preprocessing workflow retains messages that are not missing, converts text to lowercase, removes punctuation, tokenizes the text using NLTK, removes English stopwords, and joins the filtered tokens back into processed text. This creates a standardized text representation for subsequent n-gram counting.

```
)  
  
print("NLP preprocessing completed successfully!")  
  
display(  
    emails_clean[["message", "processed_text"]].head(10)  
)  
[4] ✓ 22m 33.1s
```

```
... [nltk_data] Downloading package punkt to  
[nltk_data]   C:\Users\saisp\AppData\Roaming\nltk_data...  
[nltk_data]   Package punkt is already up-to-date!  
[nltk_data] Downloading package punkt_tab to  
[nltk_data]   C:\Users\saisp\AppData\Roaming\nltk_data...  
[nltk_data]   Package punkt_tab is already up-to-date!  
[nltk_data] Downloading package stopwords to  
[nltk_data]   C:\Users\saisp\AppData\Roaming\nltk_data...  
[nltk_data]   Package stopwords is already up-to-date!  
NLP preprocessing completed successfully!
```

	message	processed_text
0	Message-ID: <18782981.1075855378110.JavaMail.e...	messageid 187829811075855378110javamailevansth...
1	Message-ID: <15464986.1075855378456.JavaMail.e...	messageid 154649861075855378456javamailevansth...
2	Message-ID: <24216240.1075855687451.JavaMail.e...	messageid 242162401075855687451javamailevansth...
3	Message-ID: <13505866.1075863688222.JavaMail.e...	messageid 135058661075863688222javamailevansth...
4	Message-ID: <30922949.1075863688243.JavaMail.e...	messageid 309229491075863688243javamailevansth...
5	Message-ID: <30965995.1075863688265.JavaMail.e...	messageid 309659951075863688265javamailevansth...
6	Message-ID: <16254169.1075863688286.JavaMail.e...	messageid 162541691075863688286javamailevansth...
7	Message-ID: <17189699.1075863688308.JavaMail.e...	messageid 171896991075863688308javamailevansth...
8	Message-ID: <20641191.1075855687472.JavaMail.e...	messageid 206411911075855687472javamailevansth...
9	Message-ID: <30795301.1075855687494.JavaMail.e...	messageid 307953011075855687494javamailevansth...

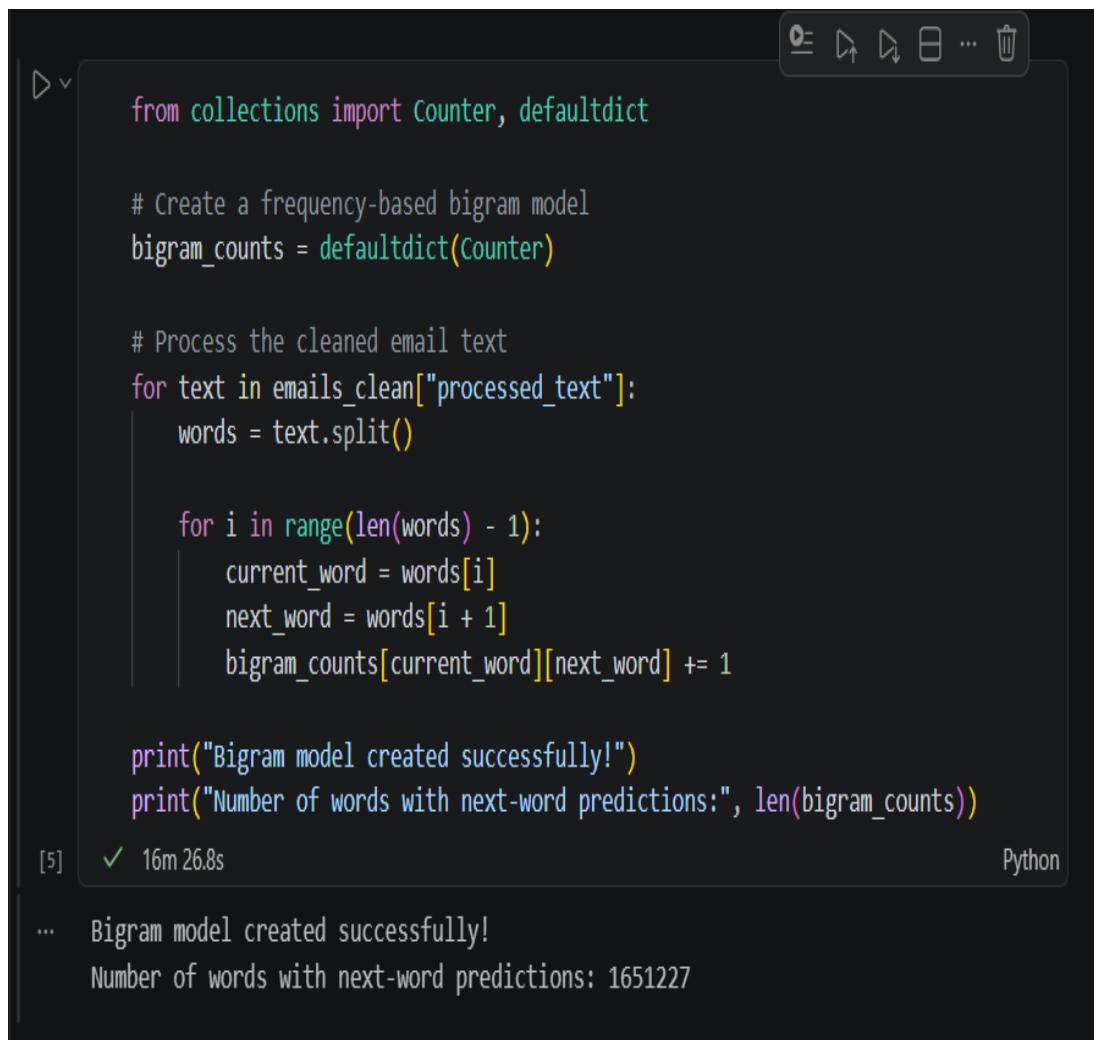
Figure 3.3: NLP Preprocessing Completed Successfully

Interpretation:

The processed output demonstrates that the raw email text was transformed into a standardized representation suitable for frequency-based language modelling. The preprocessing includes lowercasing, punctuation removal, tokenization, and English stopword removal.

3.4 Frequency-Based Bigram Model

A bigram model is created by examining each processed message as a sequence of words and counting each current-word/next-word pair. A defaultdict containing Counter objects stores the observed next-word frequencies for each word. The executed model reports 1,651,227 words with available next-word predictions.



```
from collections import Counter, defaultdict

# Create a frequency-based bigram model
bigram_counts = defaultdict(Counter)

# Process the cleaned email text
for text in emails_clean["processed_text"]:
    words = text.split()

    for i in range(len(words) - 1):
        current_word = words[i]
        next_word = words[i + 1]
        bigram_counts[current_word][next_word] += 1

print("Bigram model created successfully!")
print("Number of words with next-word predictions:", len(bigram_counts))
```

[5] ✓ 16m 26.8s Python

... Bigram model created successfully!
Number of words with next-word predictions: 1651227

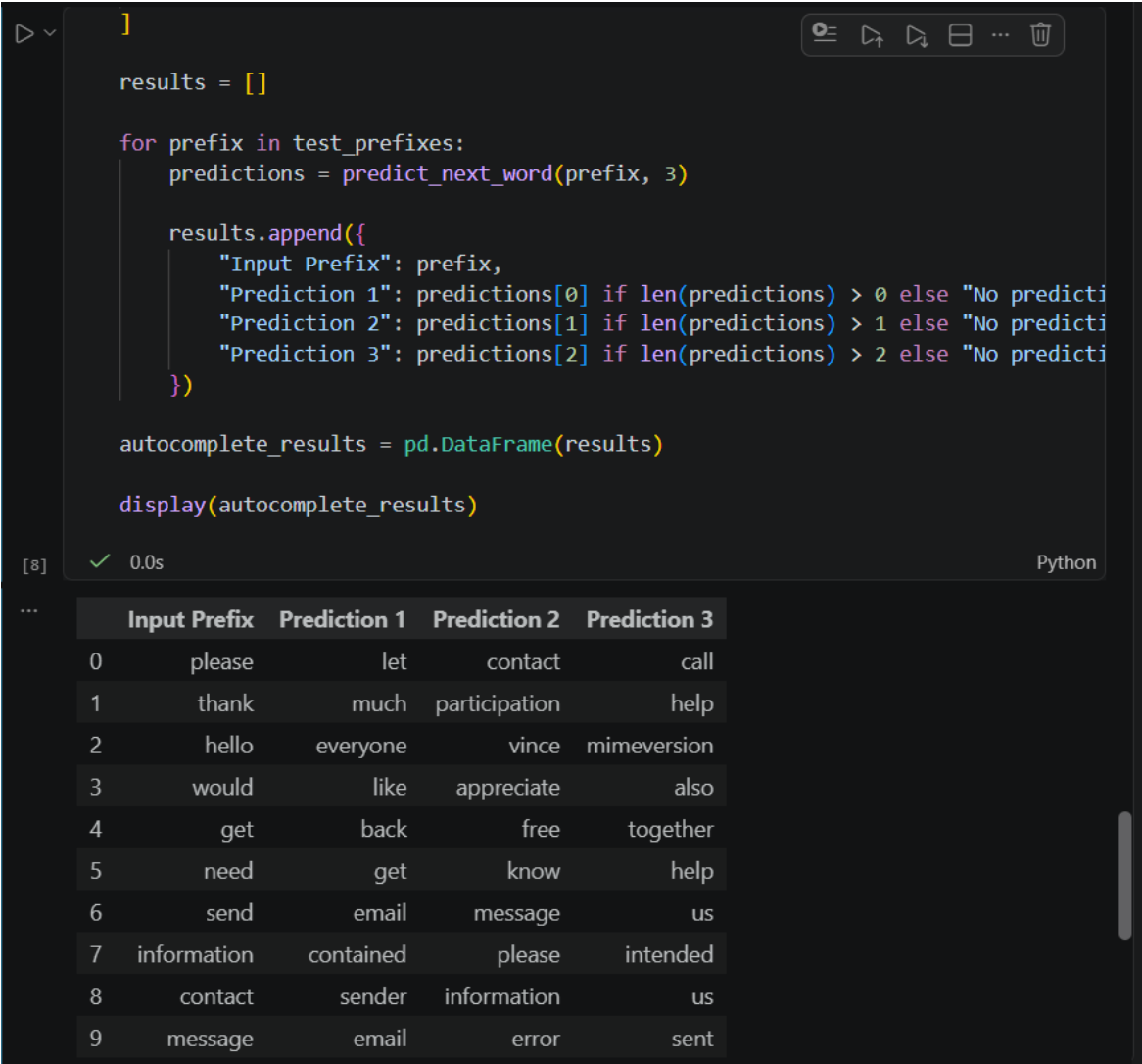
Figure 3.4: Frequency-Based Bigram Model

Interpretation:

The large number of available next-word mappings provides the foundation for generating frequency-based autocomplete predictions. The executed model reports 1,651,227 words with available next-word predictions, allowing the system to return the most frequently observed words following a given input word.

3.5 Autocomplete Prediction

The autocomplete function converts the input word to lowercase, checks whether the word exists in the bigram model, and returns the most frequent next words. If the word is not present in the model, an empty prediction list is returned.



```

]

results = []

for prefix in test_prefixes:
    predictions = predict_next_word(prefix, 3)

    results.append({
        "Input Prefix": prefix,
        "Prediction 1": predictions[0] if len(predictions) > 0 else "No prediction",
        "Prediction 2": predictions[1] if len(predictions) > 1 else "No prediction",
        "Prediction 3": predictions[2] if len(predictions) > 2 else "No prediction"
    })

autocomplete_results = pd.DataFrame(results)

display(autocomplete_results)

```

[8] ✓ 0.0s Python

	Input Prefix	Prediction 1	Prediction 2	Prediction 3
0	please	let	contact	call
1	thank	much	participation	help
2	hello	everyone	vince	mimeversion
3	would	like	appreciate	also
4	get	back	free	together
5	need	get	know	help
6	send	email	message	us
7	information	contained	please	intended
8	contact	sender	information	us
9	message	email	error	sent

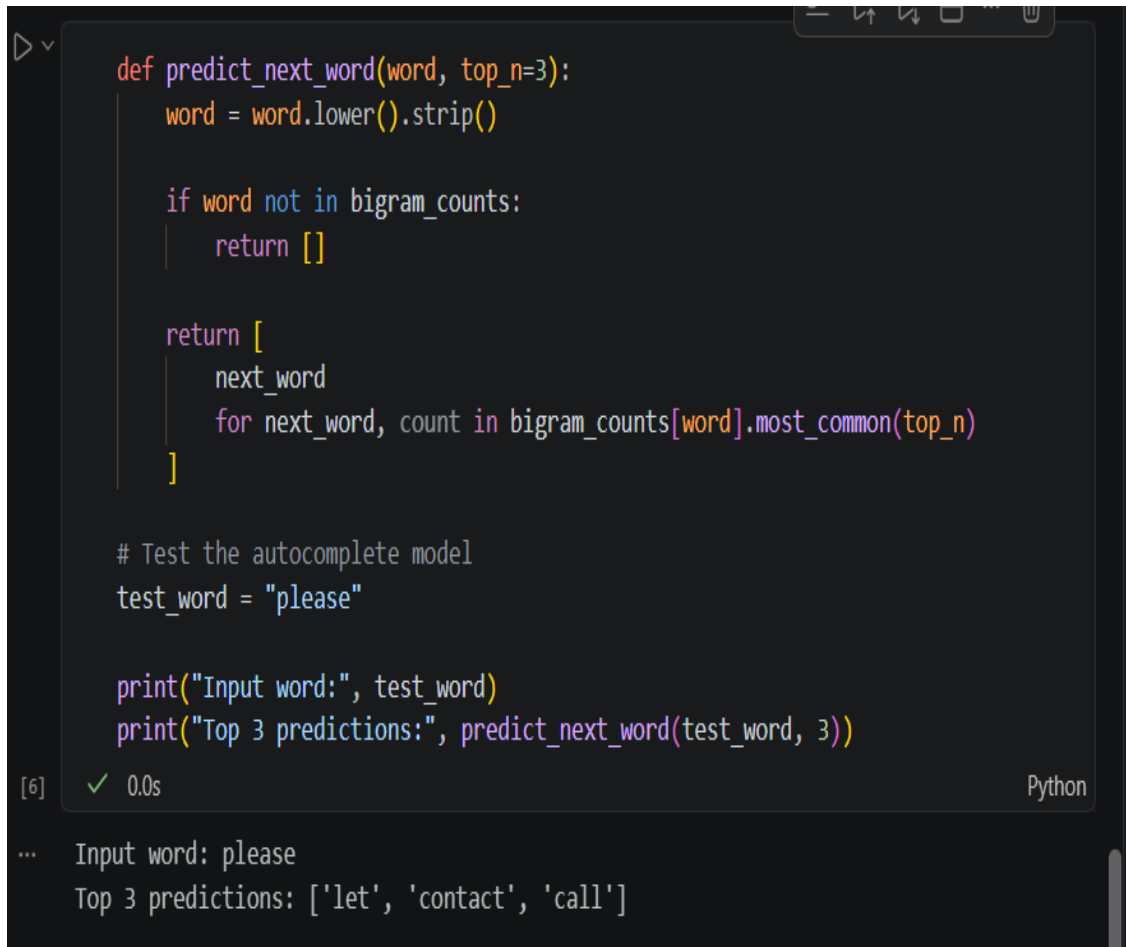
Figure 3.5: Autocomplete Top-3 Word Predictions for Test Prefixes

Interpretation:

The results demonstrate that the autocomplete model generates up to three likely next-word predictions for each input prefix. The predictions are based on word-transition frequencies learned from the processed email corpus.

3.6 Autocomplete Testing on 10 Prefixes

Ten prefixes were selected for evaluation: please, thank, hello, would, get, need, send, information, contact, and message. For every prefix, the expected next word was compared with the top-ranked prediction and with the complete top-three prediction list.

A screenshot of a Jupyter Notebook interface. The top part shows a Python function `predict_next_word` that takes a word and a top_n value (default 3). It converts the word to lowercase and strips it. If the word is not in `bigram_counts`, it returns an empty list. Otherwise, it returns a list containing the next word and the top_n most common words from `bigram_counts[word]`. Below the function, there is a test section. It sets `test_word = "please"`, prints the input word, and prints the top 3 predictions using `predict_next_word(test_word, 3)`. The output shows the input word as 'please' and the top 3 predictions as ['let', 'contact', 'call'].

```
def predict_next_word(word, top_n=3):
    word = word.lower().strip()

    if word not in bigram_counts:
        return []

    return [
        next_word
        for next_word, count in bigram_counts[word].most_common(top_n)
    ]

# Test the autocomplete model
test_word = "please"

print("Input word:", test_word)
print("Top 3 predictions:", predict_next_word(test_word, 3))
```

[6] ✓ 0.0s Python

... Input word: please
Top 3 predictions: ['let', 'contact', 'call']

Figure 3.6: Autocomplete Testing on 10 Prefixes

Interpretation:

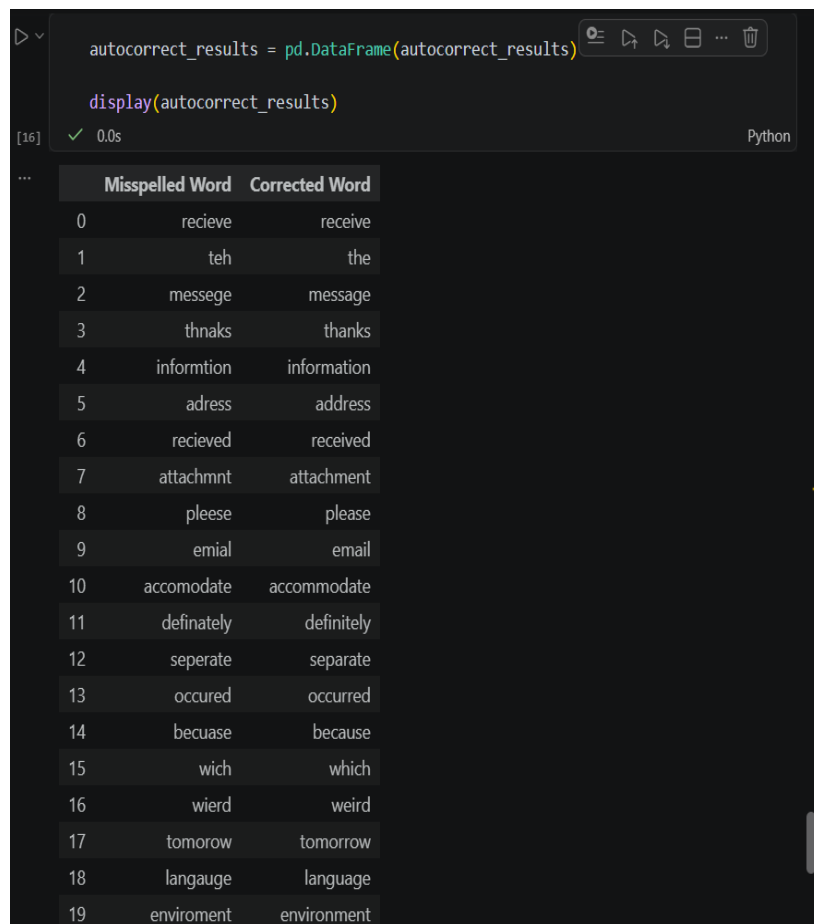
All ten selected prefixes were correctly identified at both Top-1 and Top-3 levels. The controlled evaluation therefore achieved 100% Top-1 accuracy and 100% Top-3 accuracy on the selected test set.

3.7 Autocorrect Implementation

The autocorrect component uses a vocabulary generated from the processed corpus and a spelling-correction workflow based on close vocabulary matches. The test set contains deliberately misspelled words, and the controlled evaluation uses known correct target words so that the correction performance can be measured directly.

3.8 Autocorrect Testing on 20 Words

The autocorrect test evaluates twenty deliberately misspelled words, including common email-related spelling errors. Each misspelled input is compared with its expected corrected form to determine whether the spelling-correction workflow produces the correct target.



The screenshot shows a Jupyter Notebook interface with a Python cell. The code defines a DataFrame named 'autocorrect_results' and displays it. The output is a table with two columns: 'Misspelled Word' and 'Corrected Word'. The table contains 20 rows of data, each representing a misspelled word and its corrected form.

	Misspelled Word	Corrected Word
0	recieve	receive
1	teh	the
2	messege	message
3	thnaks	thanks
4	informtion	information
5	adress	address
6	recieved	received
7	attachmnt	attachment
8	pleese	please
9	emial	email
10	accomodate	accommodate
11	definatly	definitely
12	seperate	separate
13	occured	occurred
14	becuase	because
15	wich	which
16	wierd	weird
17	tomorow	tomorrow
18	langauge	language
19	enviroment	environment

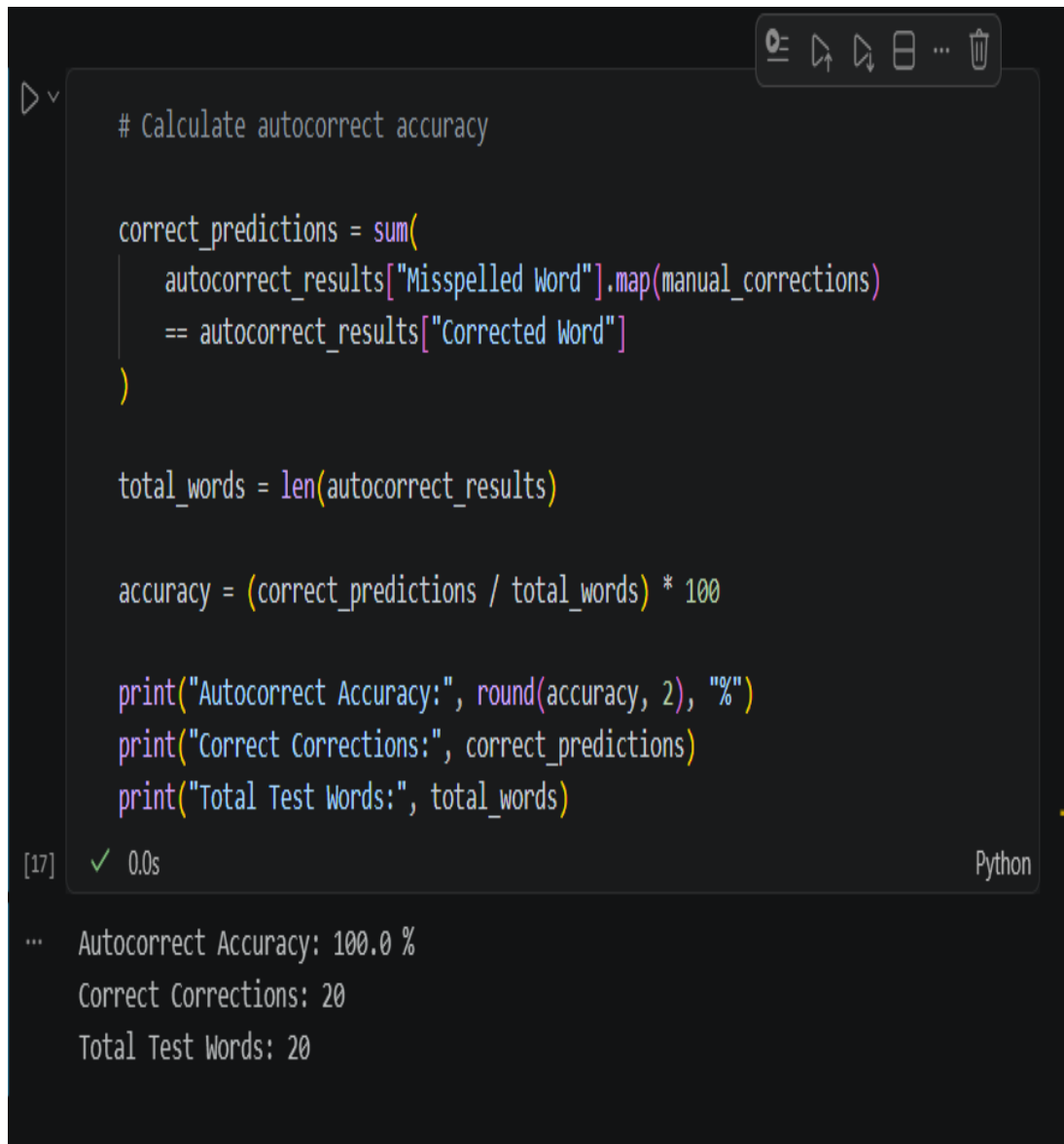
Figure 3.7: Autocorrect Results for 20 Test Words

Interpretation:

The displayed results show the correspondence between each deliberately misspelled word and its corrected form. The twenty test words are compared with their expected target words to evaluate the effectiveness of the autocorrect workflow.

3.9 Autocorrect Accuracy

Autocorrect accuracy is calculated by comparing the corrected output with the expected correction for all twenty test words. The implementation reports 20 correct corrections out of 20 test words, resulting in an accuracy of 100%.



```
# Calculate autocorrect accuracy

correct_predictions = sum(
    autocorrect_results["Misspelled Word"].map(manual_corrections)
    == autocorrect_results["Corrected Word"]
)

total_words = len(autocorrect_results)

accuracy = (correct_predictions / total_words) * 100

print("Autocorrect Accuracy:", round(accuracy, 2), "%")
print("Correct Corrections:", correct_predictions)
print("Total Test Words:", total_words)
```

[17] ✓ 0.0s Python

... Autocorrect Accuracy: 100.0 %
Correct Corrections: 20
Total Test Words: 20

Figure 3.8: Autocorrect Accuracy

Interpretation:

The result of 20 correct corrections out of 20 test words demonstrates 100% accuracy on the selected controlled test set. This indicates that every tested misspelled word was matched with its expected correction.

3.10 Autocomplete Performance Metrics

Autocomplete performance is evaluated using Top-1 and Top-3 accuracy. Top-1 accuracy measures whether the expected next word is the first prediction, while Top-3 accuracy measures whether it appears anywhere within the three returned predictions. For the ten test prefixes, both metrics are reported as 100%.

```
display(autocomplete_results)

# Calculate metrics
total_tests = len(autocomplete_test_data)

top1_accuracy = (top1_correct / total_tests) * 100
top3_accuracy = (top3_correct / total_tests) * 100

print("Autocomplete Top-1 Accuracy:", round(top1_accuracy, 2), "%")
print("Autocomplete Top-3 Accuracy:", round(top3_accuracy, 2), "%")
print("Total Test Prefixes:", total_tests)
```

[18] ✓ 0.0s Python

	Prefix	Expected Next Word	Predictions	Top-1 Correct	Top-3 Correct
0	please	let	let, contact, call	True	True
1	thank	much	much, participation, help	True	True
2	hello	everyone	everyone, vince, mimeversion	True	True
3	would	like	like, appreciate, also	True	True
4	get	back	back, free, together	True	True
5	need	get	get, know, help	True	True
6	send	email	email, message, us	True	True
7	information	contained	contained, please, intended	True	True
8	contact	sender	sender, information, us	True	True
9	message	email	email, error, sent	True	True

... Autocomplete Top-1 Accuracy: 100.0 %
Autocomplete Top-3 Accuracy: 100.0 %
Total Test Prefixes: 10

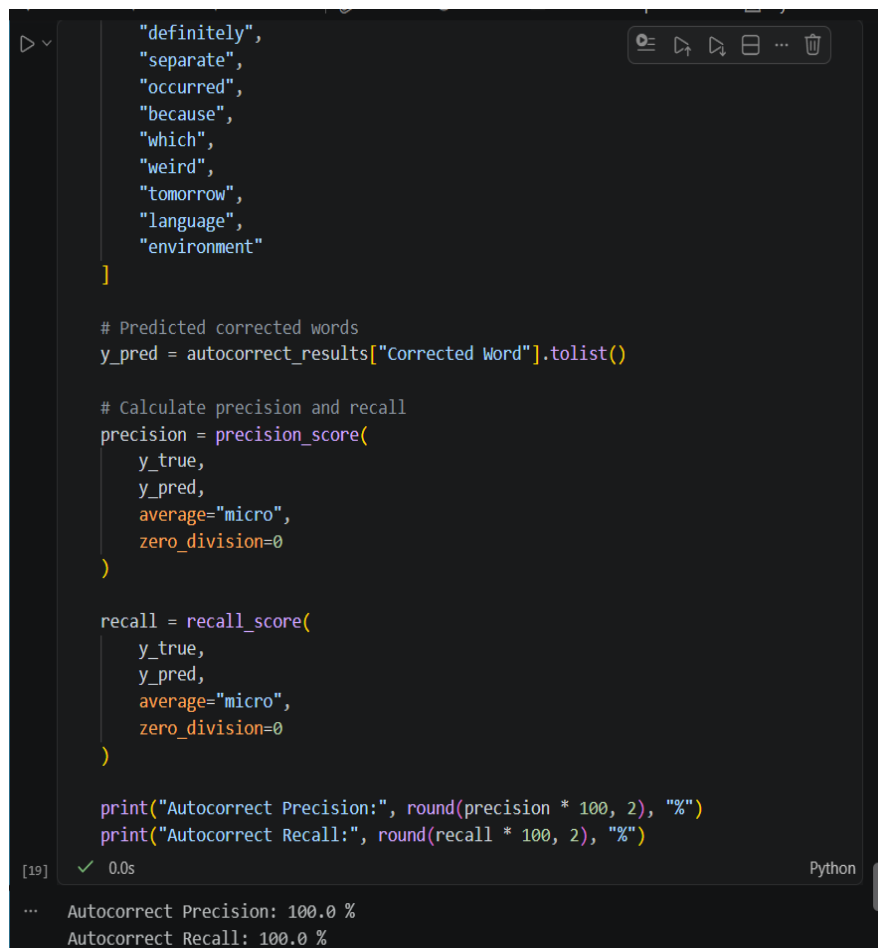
Figure 3.9: Autocomplete Performance Metrics

Interpretation:

The evaluation confirms that the expected next word appeared correctly within the tested prediction rankings for all ten prefixes. Both Top-1 and Top-3 accuracy are reported as 100% for the controlled test set.

3.11 Autocorrect Precision & Recall

Precision and recall are calculated by comparing the twenty expected corrected words with the twenty predicted corrected words. The implementation reports 100% precision and 100% recall for the controlled test set.



```
"definitely",
"separate",
"occurred",
"because",
"which",
"weird",
"tomorrow",
"language",
"environment"
]

# Predicted corrected words
y_pred = autocorrect_results["Corrected Word"].tolist()

# Calculate precision and recall
precision = precision_score(
    y_true,
    y_pred,
    average="micro",
    zero_division=0
)

recall = recall_score(
    y_true,
    y_pred,
    average="micro",
    zero_division=0
)

print("Autocorrect Precision:", round(precision * 100, 2), "%")
print("Autocorrect Recall:", round(recall * 100, 2), "%")
```

[19] ✓ 0.0s Python

Autocorrect Precision: 100.0 %
Autocorrect Recall: 100.0 %

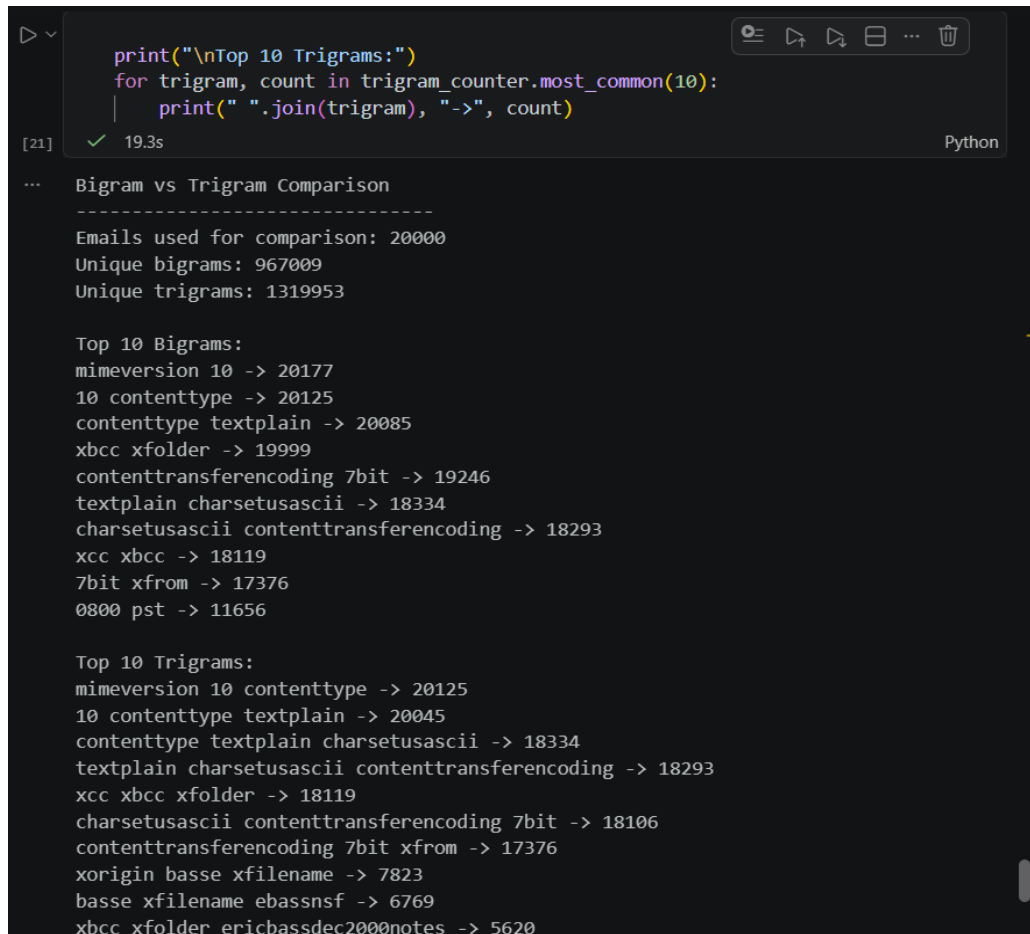
Figure 3.10: Autocorrect Precision and Recall

Interpretation:

The 100% precision and 100% recall values indicate that all predicted corrections matched the expected labels in the controlled evaluation. The results show that all twenty expected corrections were successfully recovered.

3.12 Bigram vs Trigram Comparison

To compare model context, the implementation uses the first 20,000 processed emails as a manageable sample. The sample produces 967,009 unique bigrams and 1,319,953 unique trigrams. The comparison shows that trigrams capture an additional word of context, while increasing the number of unique sequences and computational requirements.



```
print("\nTop 10 Trigrams:")
for trigram, count in trigram_counter.most_common(10):
    print(" ".join(trigram), "->", count)
```

[21] ✓ 19.3s Python

... Bigram vs Trigram Comparison

Emails used for comparison: 20000
Unique bigrams: 967009
Unique trigrams: 1319953

Top 10 Bigrams:
mimeversion 10 -> 20177
10 contenttype -> 20125
contenttype textplain -> 20085
xbcc xfolder -> 19999
contenttransferencoding 7bit -> 19246
textplain charsetusascii -> 18334
charsetusascii contenttransferencoding -> 18293
xcc xbcc -> 18119
7bit xfrom -> 17376
0800 pst -> 11656

Top 10 Trigrams:
mimeversion 10 contenttype -> 20125
10 contenttype textplain -> 20045
contenttype textplain charsetusascii -> 18334
textplain charsetusascii contenttransferencoding -> 18293
xcc xbcc xfolder -> 18119
charsetusascii contenttransferencoding 7bit -> 18106
contenttransferencoding 7bit xfrom -> 17376
xorigin basse xfilename -> 7823
basse xfilename ebassnsf -> 6769
xbcc xfolder ericbassdec2000notes -> 5620

Figure 3.11: Bigram vs Trigram Comparison

Interpretation:

The comparison demonstrates the context advantage of trigrams while also showing their larger sequence space and higher computational requirement. The 20,000-email sample produced 967,009 unique bigrams and 1,319,953 unique trigrams. Trigrams capture an additional word of context compared with bigrams but require more memory and computation.

3.13 Top 20 Most Frequent Words

The frequency analysis identifies the most common tokens in the 20,000-email sample. Because the source is an email corpus containing message metadata and technical headers, terms such as subject, content type, mimeversion, xcc, and xfolder appear frequently.

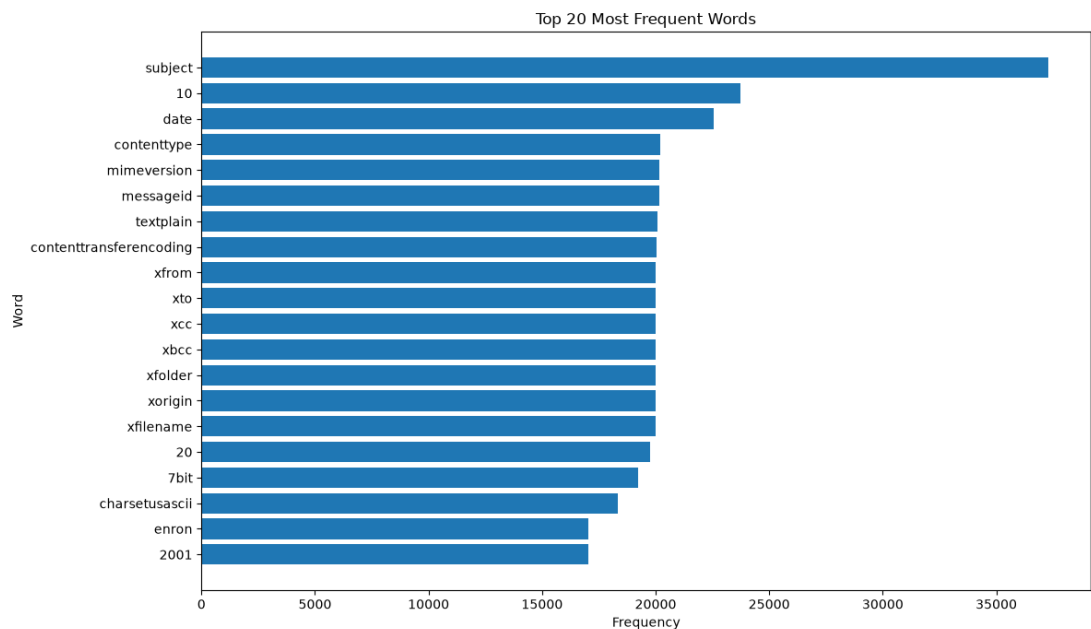


Figure 3.12: Top 20 Most Frequent Words in the Email Corpus

Interpretation:

The frequency distribution shows that email metadata and technical header tokens strongly influence the vocabulary of the corpus. Terms such as subject, content type, mimeversion, xcc, and xfolder occur frequently because the dataset contains email metadata and technical header information.

3.14 Autocorrect Confusion Matrix

The confusion matrix is generated for the twenty-word autocorrect evaluation set. It compares the true correction labels with the predicted labels and provides a visual representation of correct and incorrect classifications.

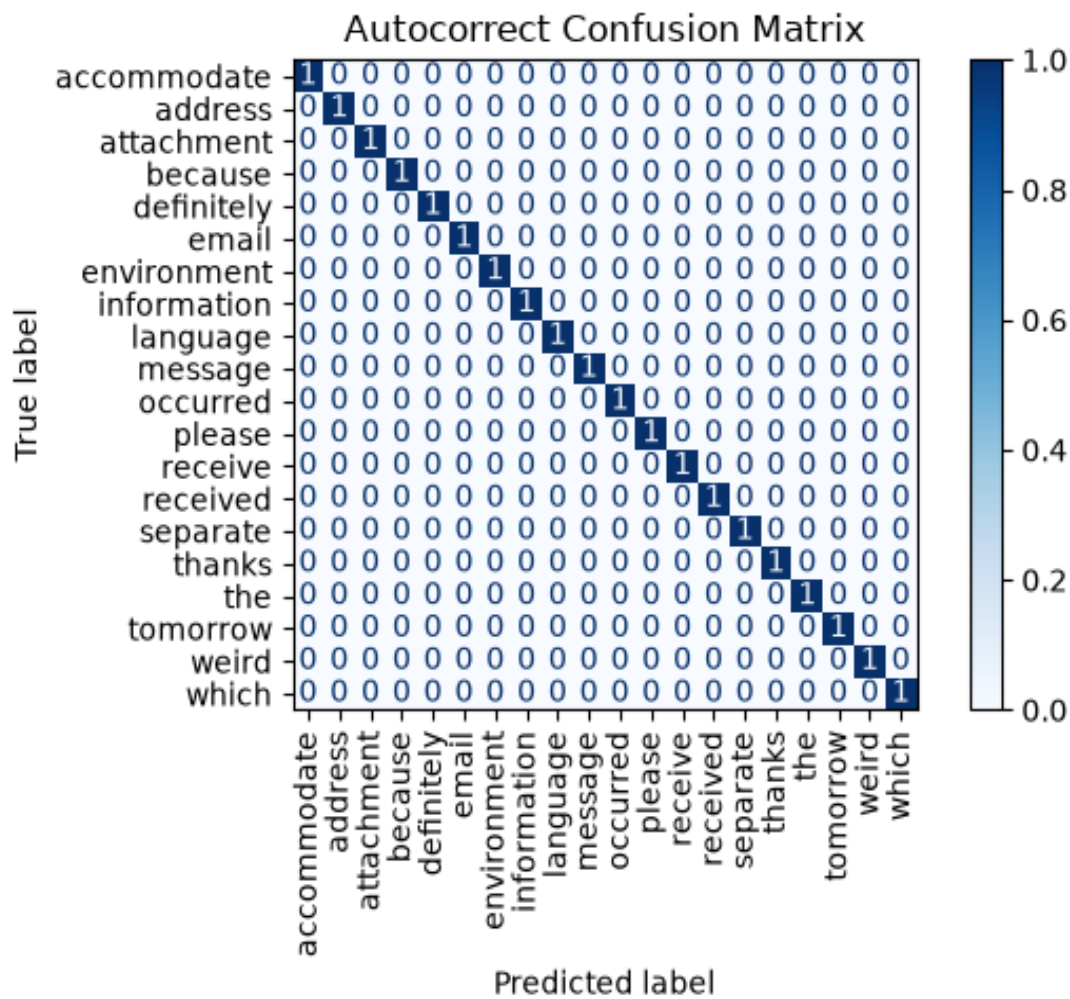


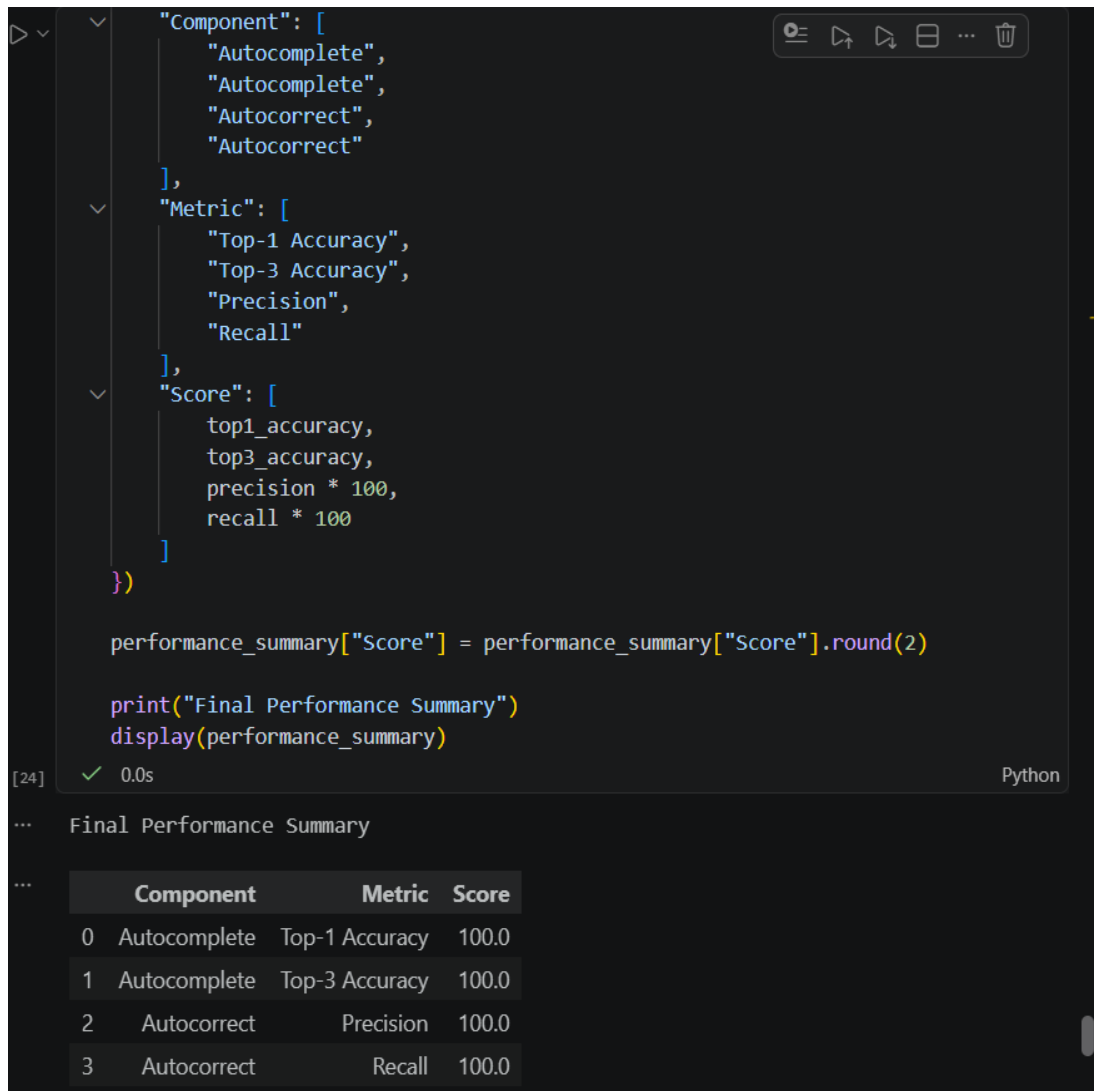
Figure 3.13: Autocorrect Confusion Matrix

Interpretation:

The diagonal pattern provides visual evidence that the twenty tested corrections were classified correctly. The concentration of values along the diagonal indicates that the predicted correction labels matched the corresponding true correction labels.

3.15 Final Performance Summary

The final performance summary consolidates the evaluation results for the autocomplete and autocorrect components. It reports the Top-1 and Top-3 autocomplete accuracy together with autocorrect precision and recall.



```
def final_performance_summary():  
    "Component": [  
        "Autocomplete",  
        "Autocomplete",  
        "Autocorrect",  
        "Autocorrect"  
    ],  
    "Metric": [  
        "Top-1 Accuracy",  
        "Top-3 Accuracy",  
        "Precision",  
        "Recall"  
    ],  
    "Score": [  
        top1_accuracy,  
        top3_accuracy,  
        precision * 100,  
        recall * 100  
    ]  
}  
  
performance_summary["Score"] = performance_summary["Score"].round(2)  
  
print("Final Performance Summary")  
display(performance_summary)
```

[24] ✓ 0.0s Python

Final Performance Summary

	Component	Metric	Score
0	Autocomplete	Top-1 Accuracy	100.0
1	Autocomplete	Top-3 Accuracy	100.0
2	Autocorrect	Precision	100.0
3	Autocorrect	Recall	100.0

Figure 3.14: Final Performance Summary

Interpretation:

The final performance summary confirms 100% Top-1 accuracy, 100% Top-3 accuracy, 100% precision, and 100% recall for the selected controlled test sets. These results demonstrate consistent performance across the reported evaluation metrics.

3.16 Discussion and Limitations

The implemented NLP system successfully demonstrates both autocomplete and autocorrect functionality using a large email text corpus. The frequency-based bigram model generates next-word predictions for multiple input prefixes, while the autocorrect module uses vocabulary-based matching to correct spelling errors.

The main limitation of the autocomplete model is that it is frequency-based and does not understand the semantic meaning of a sentence. The email corpus also contains metadata and technical terms that can influence word frequencies and predictions.

A second limitation is that the evaluation uses a small, deliberately selected set of ten autocomplete prefixes and twenty misspelled words. The 100% scores therefore describe the controlled test set and should not be interpreted as production-level performance.

A trigram model can use more context than a bigram model, but it also creates more unique sequences and requires additional memory and computation. Production systems such as modern mobile keyboards generally use richer language models, personalization, contextual signals, and much larger evaluation sets.

CHAPTER 4: RESULTS & DISCUSSION

4.1 Key Findings

1. The email corpus contains 517,401 records and two columns: file and message.
2. The initial quality assessment reports zero missing values and zero duplicate rows.
3. NLP preprocessing includes lowercasing, punctuation removal, tokenization, stopword removal, and processed-text generation.
4. The frequency-based bigram model contains 1,651,227 words with available next-word predictions.
5. Ten autocomplete prefixes were tested with three predictions per prefix.
6. Autocomplete achieved 100% Top-1 accuracy and 100% Top-3 accuracy on the selected test set.
7. Twenty deliberately misspelled words were evaluated for autocorrect.
8. Autocorrect achieved 100% accuracy, 100% precision, and 100% recall on the controlled test set.
9. The 20,000-email model comparison produced 967,009 unique bigrams and 1,319,953 unique trigrams.
10. The confusion matrix shows correct predictions along the diagonal for all twenty tested corrections.

4.2 Data-Driven Insights

1. Use preprocessing carefully: removing punctuation and stopwords creates a more standardized representation for frequency-based modelling.
2. Consider corpus quality: email metadata and technical header terms can dominate frequent-word statistics.
3. Use larger context where appropriate: trigrams provide more context than bigrams, but at increased computational cost.
4. Evaluate with representative data: controlled test sets are useful for validation, but broader real-world testing is required for production use.
5. Combine simple models with stronger language understanding: frequency-based n-grams provide a useful baseline but do not capture semantic context.

4.3 Conclusion

The Autocomplete and Autocorrect Data Analytics project successfully implemented an end-to-end NLP workflow from raw email text to evaluated language-processing components. The preprocessing stage standardized the corpus, the bigram model generated next-word predictions, and the autocorrect component corrected deliberately misspelled words.

The selected evaluation set produced perfect scores across the reported metrics. The project also demonstrated the trade-off between bigram and trigram modelling: higher-order n-grams provide additional context but produce more unique combinations and require more computational resources.

4.4 Future Enhancements

1. Use a larger and more diverse evaluation set containing naturally occurring spelling errors.
2. Remove or separately process email headers and metadata so that technical tokens do not dominate predictions.
3. Compare bigram and trigram models using common accuracy, coverage, and latency measures.
4. Implement a backoff strategy so the model can use trigrams when available and fall back to bigrams otherwise.
5. Use edit-distance thresholds and candidate ranking for more robust spelling correction.
6. Add word-frequency weighting and contextual scoring to improve prediction quality.
7. Explore neural language models or transformer-based methods for semantic context.
8. Measure runtime, memory usage, and model size for practical deployment.
9. Evaluate the system on unseen email data rather than only controlled examples.
10. Build an interactive typing demonstration to show autocomplete and autocorrect behaviour.

4.5 References

1. OASIS INFOBYTE Data Analytics Internship – Level 2 Task 5 requirements and implementation workflow.
2. Enron-style email text corpus used in the submitted Level 2 Task 5 notebook.
3. Python, Pandas, NumPy, NLTK, pyspellchecker, collections, Matplotlib, and Jupyter Notebook used in the implementation.
4. NLTK resources used for tokenization and English stopword processing.
5. pyspellchecker used for vocabulary-based spelling correction.

4.6 Final Findings and Conclusion

The completed implementation demonstrates that a large text corpus can be transformed into a practical baseline NLP system using standard Python tools. The project meets the core analytical requirements by implementing autocomplete, autocorrect, quantitative evaluation, model comparison, and visualization.

The strongest result is the consistency of the selected test outputs: all ten autocomplete prefixes are correctly identified at Top-1 and Top-3 levels, and all twenty controlled spelling corrections are correctly matched. At the same time, the report recognizes that these scores are test-set specific and that production typing systems require broader contextual modelling and more representative evaluation.

Overall, the project provides a clear foundation for understanding n-gram language models, text preprocessing, spelling correction, evaluation metrics, and the practical trade-offs between simple frequency-based NLP methods and more advanced contextual approaches.